

BloodLink — Test Cases Specification

Field	Value
Project	BloodLink — Blood Bank Management System
Version	2.0
Date	May 1, 2026
Author	BloodLink QA Team
Tools	Jest 30 (Unit), Cypress 14 (System E2E)

1. Introduction

This document provides the complete test case specification for the BloodLink platform, organized by test level: **Unit Tests**, **Integration Tests**, and **System (E2E) Tests**. Each test case includes an ID, description, preconditions, steps, expected results, and priority level.

Priority Legend

Symbol	Level	Description
●	Critical	Must pass before any release
●	High	Required for sprint completion
●	Medium	Should pass, deferrable
●	Low	Nice to have

2. Unit Test Cases

2.1 Auth Module (auth.service.spec.ts)

ID	Title	Priority	Preconditions	Steps	Expected Result
UT-AUTH-01	Valid login	●	User exists with hashed password	Call <code>login(email, password)</code> with correct credentials	Returns <code>{ access_token, user }</code> with no password field in user object
UT-AUTH-02	Wrong password	●	User exists	Call <code>login(email, wrongPassword)</code>	Throws <code>UnauthorizedException</code>
UT-AUTH-03	Non-existent email	●	No user with given email	Call <code>login(unknownEmail, pass)</code>	Throws <code>UnauthorizedException</code>
UT-AUTH-04	Get profile	●	User exists	Call <code>getProfile(userId)</code>	Returns sanitized user object (no password)
UT-AUTH-05	Profile not found	●	No user with given ID	Call <code>getProfile(invalidId)</code>	Throws <code>UnauthorizedException</code>

ID	Title	Priority	Preconditions	Steps	Expected Result
UT-AUTH-06	Bcrypt verification	●	User exists	Call <code>login()</code>	<code>bcrypt.compare()</code> is called with plaintext input and stored hash
UT-AUTH-07	JWT payload	●	User exists	Call <code>login()</code>	JWT is signed with payload <code>{ sub, email, role }</code>

2.2 Donors Module — Blood Compatibility Matrix (`donors.service.spec.ts`)

ID	Title	Priority	Input Blood Type	Expected Compatible Donors
UT-COMPAT-01	O+ compatibility	●	O+	O+, O-
UT-COMPAT-02	O- compatibility	●	O-	O- only
UT-COMPAT-03	A+ compatibility	●	A+	A+, A-, O+, O-
UT-COMPAT-04	A- compatibility	●	A-	A-, O-
UT-COMPAT-05	B+ compatibility	●	B+	B+, B-, O+, O-
UT-COMPAT-06	B- compatibility	●	B-	B-, O-
UT-COMPAT-07	AB+ universal recipient	●	AB+	All 8 blood types
UT-COMPAT-08	AB- compatibility	●	AB-	A-, B-, AB-, O-

2.3 Donors Module — CRUD & Filters

ID	Title	Priority	Preconditions	Steps	Expected Result
UT-DONOR-01	Create with defaults	●	None	Call <code>create(dto)</code> without optional fields	<code>reliability=50</code> , <code>available=true</code> , <code>eligible=true</code> , <code>totalDonations=0</code>
UT-DONOR-02	Find non-existent	●	No donor with ID	Call <code>findOne(invalidId)</code>	Throws <code>NotFoundException</code>
UT-DONOR-03	Remove donor	●	Donor exists	Call <code>remove(id)</code>	Returns <code>{ deleted: true }</code>
UT-DONOR-04	Toggle true → false	●	Donor <code>available=true</code>	Call <code>toggleAvailability(id)</code>	Returns donor with <code>available=false</code>
UT-DONOR-05	Toggle false → true	●	Donor <code>available=false</code>	Call <code>toggleAvailability(id)</code>	Returns donor with <code>available=true</code>
UT-DONOR-06	Filter available+eligible	●	None	Call <code>findCompatible(type)</code>	Query includes <code>available=true AND eligible=true</code>
UT-DONOR-07	Sort by reliability	●	None	Call <code>findCompatible(type)</code>	Results ordered by <code>reliability DESC</code>

2.4 Inventory Module — Status Boundaries (`inventory.service.spec.ts`)

ID	Title	Priority	Input Units	Expected Status	Expected Critical
UT-INV-01	Critical boundary (15)	●	15	CRITICAL	true

ID	Title	Priority	Input Units	Expected Status	Expected Critical
UT-INV-02	Critical boundary (30)	●	30	CRITICAL	true
UT-INV-03	Warning lower (31)	●	31	WARNING	false
UT-INV-04	Warning mid (50)	●	50	WARNING	false
UT-INV-05	Warning upper (80)	●	80	WARNING	false
UT-INV-06	Healthy boundary (81)	●	81	HEALTHY	false
UT-INV-07	Healthy (100)	●	100	HEALTHY	false

2.5 Emergencies Module (emergencies.service.spec.ts)

ID	Title	Priority	Preconditions	Steps	Expected Result
UT-EMRG-01	Find all active	●	Active emergencies exist	Call <code>findAll()</code>	Returns only ACTIVE, ordered by <code>createdAt DESC</code>
UT-EMRG-02	Find non-existent	●	No emergency with ID	Call <code>findOne(invalidId)</code>	Throws <code>NotFoundException</code>
UT-EMRG-03	Create with defaults	●	None	Call <code>create(dto)</code> without distance	<code>distance=0</code>
UT-EMRG-04	Resolve emergency	●	Active emergency	Call <code>resolve(id)</code>	<code>status=RESOLVED</code>
UT-EMRG-05	Remove emergency	●	Emergency exists	Call <code>remove(id)</code>	Returns <code>{ deleted: true }</code>

2.6 Matching Engine

ID	Title	Priority	Preconditions	Steps	Expected Result
UT-MATCH-01	Response structure	●	1 compatible donor	Call <code>matchDonors(id)</code>	Returns <code>{ totalCompatible, highReliability, donors[] }</code> with score, distance, eta per donor
UT-MATCH-02	High reliability count	●	3 donors (reliability: 95, 80, 99)	Call <code>matchDonors(id)</code>	<code>highReliability = 2</code> (counts donors with reliability ≥ 90)
UT-MATCH-03	Score sort order	●	2 donors with different scores	Call <code>matchDonors(id)</code>	Donors sorted by score descending
UT-MATCH-04	Top-10 cap	●	15 compatible donors	Call <code>matchDonors(id)</code>	<code>donors.length = 10</code> , <code>totalCompatible = 15</code>
UT-MATCH-05	Zero donors	●	0 compatible donors	Call <code>matchDonors(id)</code>	<code>totalCompatible = 0</code> , <code>donors = []</code>
UT-MATCH-06	Score formula	●	Donor with reliability=100	Call <code>matchDonors(id)</code>	Score = <code>round(100 × 0.7 + distanceScore × 0.3)</code> ∈ [70, 100]

2.7 Notifications Module (notifications.service.spec.ts)

ID	Title	Priority	Preconditions	Steps	Expected Result
UT-NOTIF-01	Find for user	●	Notifications exist for user	Call <code>findAllForUser(userId)</code>	Returns notifications ordered by <code>timestamp DESC</code>

ID	Title	Priority	Preconditions	Steps	Expected Result
UT-NOTIF-02	Mark as read		Unread notification exists	Call <code>markAsRead(id)</code>	Returns notification with <code>read = true</code>
UT-NOTIF-03	Mark — not found		No notification with ID	Call <code>markAsRead(invalidId)</code>	Throws <code>NotFoundException</code>
UT-NOTIF-04	Mark all read		Multiple unread for user	Call <code>markAllReadForUser(userId)</code>	Returns <code>{ success: true }</code> , bulk update executed
UT-NOTIF-05	Dismiss		Notification exists	Call <code>dismiss(id)</code>	Returns <code>{ deleted: true }</code>
UT-NOTIF-06	Dismiss — not found		No notification with ID	Call <code>dismiss(invalidId)</code>	Throws <code>NotFoundException</code>
UT-NOTIF-07	Clear all for user		Notifications exist	Call <code>clearAllForUser(userId)</code>	Returns <code>{ deleted: true }</code>
UT-NOTIF-08	Create notification		None	Call <code>create(data)</code>	Notification saved with correct fields

3. Integration Test Cases

Integration tests verify the full HTTP request → controller → service → repository pipeline using **Supertest** against the bootstrapped NestJS application. All controllers, pipes, and guards are active; only the database layer is mocked.

3.1 Auth API Integration

ID	Title	Priority	Method	Endpoint	Preconditions	Expected Result
IT-AUTH-01	Valid login		POST	<code>/api/auth/login</code>	Seeded admin user	201, body contains <code>access_token</code> and sanitized <code>user</code> (no password)
IT-AUTH-02	Wrong password		POST	<code>/api/auth/login</code>	User exists	401 Unauthorized
IT-AUTH-03	Unknown email		POST	<code>/api/auth/login</code>	No user	401 Unauthorized
IT-AUTH-04	Authenticated profile		GET	<code>/api/auth/me</code>	Valid JWT in header	200, sanitized user returned
IT-AUTH-05	Unauthenticated profile		GET	<code>/api/auth/me</code>	No Authorization header	401 / 403 Forbidden

3.2 Donors API Integration

ID	Title	Priority	Method	Endpoint	Preconditions	Expected Result
IT-DONOR-01	List all donors		GET	/api/donors	JWT auth	200, array of donors
IT-DONOR-02	Filter by blood type		GET	/api/donors?bloodType=0+	JWT auth	200, <code>andWhere</code> called with blood type filter
IT-DONOR-03	Search by name		GET	/api/donors?search=Mohamed	JWT auth	200, <code>andWhere</code> called with LIKE name filter
IT-DONOR-04	Create donor		POST	/api/donors	JWT auth	201, defaults applied (<code>reliability=50</code> , <code>available=true</code>)
IT-DONOR-05	Toggle availability		PATCH	/api/donors/:id/toggle-availability	JWT auth, donor exists	200, <code>available</code> field toggled
IT-DONOR-06	Delete donor		DELETE	/api/donors/:id	JWT auth, donor exists	200, { <code>deleted: true</code> }

3.3 Inventory API Integration

ID	Title	Priority	Method	Endpoint	Preconditions	Expected Result
IT-INV-01	List all inventory		GET	/api/inventory	JWT auth	200, array of items
IT-INV-02	Filter by status		GET	/api/inventory?status=Critical	JWT auth	200, <code>andWhere</code> filter applied
IT-INV-03	Create critical item		POST	/api/inventory	JWT auth, <code>units=10</code>	201, <code>status=CRITICAL</code> , <code>critical=true</code>
IT-INV-04	Create healthy item		POST	/api/inventory	JWT auth, <code>units=200</code>	201, <code>status=HEALTHY</code> , <code>critical=false</code>
IT-INV-05	Update recalculates		PATCH	/api/inventory/:id	JWT auth, existing item	200, status recalculated from new units
IT-INV-06	Delete item		DELETE	/api/inventory/:id	JWT auth, item exists	200, { <code>deleted: true</code> }

3.4 Emergencies API Integration

ID	Title	Priority	Method	Endpoint	Preconditions	Expected Result
IT-EMRG-01	List active emergencies		GET	/api/emergencies	JWT auth	200, array of ACTIVE emergencies
IT-EMRG-02	Create emergency		POST	/api/emergencies	JWT auth	201, <code>distance=0</code> default

ID	Title	Priority	Method	Endpoint	Preconditions	Expected Result
IT-EMRG-03	Run matching engine		GET	<code>/api/emergencies/:id/match</code>	JWT auth, emergency exists	200, { totalCompatible, donors[] } with scores
IT-EMRG-04	Notify donors		POST	<code>/api/emergencies/:id/notify</code>	JWT auth, emergency exists	201, { notified: N }
IT-EMRG-05	Resolve emergency		PATCH	<code>/api/emergencies/:id/resolve</code>	JWT auth, active emergency	200, status=RESOLVED
IT-EMRG-06	Delete emergency		DELETE	<code>/api/emergencies/:id</code>	JWT auth, emergency exists	200, { deleted: true }

3.5 Notifications API Integration

ID	Title	Priority	Method	Endpoint	Preconditions	Expected Result
IT-NOTIF-01	List user notifications		GET	<code>/api/notifications</code>	JWT auth	200, array ordered by timestamp DESC
IT-NOTIF-02	Mark as read		PATCH	<code>/api/notifications/:id/read</code>	JWT auth, notification exists	200, read=true
IT-NOTIF-03	Mark all read		PATCH	<code>/api/notifications/read-all</code>	JWT auth	200, { success: true }
IT-NOTIF-04	Dismiss single		DELETE	<code>/api/notifications/:id</code>	JWT auth, notification exists	200, { deleted: true }
IT-NOTIF-05	Clear all		DELETE	<code>/api/notifications/clear-all</code>	JWT auth	200, { deleted: true }

4. System (E2E) Test Cases — Cypress

System tests validate complete user workflows through the browser using Cypress. The test suite is located at `frontend/cypress/e2e/bloodlink.cy.ts`.

3.1 Test Environment

Component	Configuration
Frontend	Next.js on <code>http://localhost:3000</code>
Backend	NestJS on <code>http://localhost:3001</code>
Auth Method	Session-based login via <code>cy.session()</code> with admin credentials
Test Runner	Cypress 14 with <code>cy.session</code> for persistent auth

4.2 System Test Cases

ID	Title	Priority	Preconditions	Steps	Expected Result
E2E-SYS-01	Dashboard stats visible	●	Logged in as admin	1. Login via <code>cy.session</code> 2. Visit <code>/</code> (Dashboard)	"Platform Overview", "Emergencies", and "Inventory" are visible on the page
E2E-SYS-02	Navigate to Inventory	●	Logged in	1. Click "Inventory" in nav 2. Verify URL change	URL includes <code>/inventory</code> . "Blood Type" and "Available Units" headers are visible
E2E-SYS-03	Toggle profile menu	●	Logged in	1. Click user avatar "HA" in header	"My Profile" and "Sign Out" options are visible
E2E-SYS-04	Create emergency & match	●	Logged in on Emergencies page	1. Navigate to <code>/emergencies</code> 2. Click "+" button 3. Fill form: Hospital="Cypress Emergency Hospital", Dept="Emergency Room", Type=O-, Units=8, Urgency=Critical 4. Click "Create Emergency Request"	Success toast "Emergency Created" displayed. "Cypress Emergency Hospital" appears in the list. "Matching Engine" section loads

4.3 Auth Session Flow

```
cy.session('admin-session', () => {
  cy.visit('/login');
  cy.contains('admin@bloodlink.org').click(); // Quick-login preset
  cy.get('button[type="submit"]').click();
  cy.url().should('eq', 'http://localhost:3000/');
});
```

This shared session is reused across all tests via `beforeEach`, preventing redundant logins and ensuring authentication state persists.

5. Test Execution Summary

5.1 By Test Level

Level	Framework	Total Cases	Automated	Run Command
Unit	Jest 30 + ts-jest	53	✓ All	<code>cd backend && npm run test</code>
Unit (Coverage)	Jest 30	53	✓ All	<code>cd backend && npm run test:cov</code>
Integration	Jest 30 + Supertest	28	✓ All	<code>cd backend && npm run test:e2e</code>
System (E2E)	Cypress 14	4	✓ All	<code>cd frontend && npx cypress run</code>
Total	—	85	85	—

5.2 By Module

Module	Unit Tests	Integration Tests	System Tests	Total
Auth	7	5	1 (session)	13
Donors	15	6	0	21
Inventory	7	6	1	14
Emergencies / Matching	13	6	1	20
Notifications	9	5	0	14

Module	Unit Tests	Integration Tests	System Tests	Total
Dashboard	0	0	1	1
Profile	0	0	1	1
App Controller	1	0	0	1
Total	52	28	5	85

6. CI/CD Integration

The project uses **GitHub Actions** for continuous integration. The workflow file is located at `.github/workflows/test.yml`.

Pipeline Steps

Step	Action	Duration
Checkout Repository	<code>actions/checkout@v4</code>	~1s
Setup Node.js	<code>actions/setup-node@v4</code> (Node 20)	~2s
Install Dependencies	<code>npm ci</code> (with cache)	~8s
Run Unit Tests	<code>npm run test:cov</code> (Jest + Coverage)	~7s
Verify Coverage	Report generated in <code>backend/coverage/</code>	~0s

Trigger Events

- Push to `main` branch
- Pull requests targeting `main`

7. Traceability Matrix

Requirement	Unit Tests	Integration Tests	System Tests
User Authentication	UT-AUTH-01 – 07	IT-AUTH-01 – 05	E2E-SYS-01 (session)
Blood Compatibility	UT-COMPAT-01 – 08	IT-EMRG-03	E2E-SYS-04 (match)
Inventory Status	UT-INV-01 – 07	IT-INV-01 – 06	E2E-SYS-02
Donor Management	UT-DONOR-01 – 07	IT-DONOR-01 – 06	—
Emergency Matching	UT-MATCH-01 – 06	IT-EMRG-03 – 04	E2E-SYS-04
Notifications	UT-NOTIF-01 – 08	IT-NOTIF-01 – 05	—
Dashboard Overview	—	—	E2E-SYS-01
User Profile	—	—	E2E-SYS-03

8. Approval

Role	Name	Signature	Date
QA Lead			
Dev Lead			

Role	Name	Signature	Date
Project Manager			